Latent MLP — Stage 2 Architecture

Mapping Physical Parameters to Latent Codes

Project 2: Nonlinear Data-Driven Reduced Order Models

1 Purpose and Context

The Latent MLP is the second stage in a two-stage parametric reduced-order model (ROM) pipeline:

1. **Stage 1 — Convolutional Autoencoder (CAE):** A trained CAE compresses high-dimensional spatial fields $\mathbf{u} \in \mathbb{R}^{C \times H \times W}$ into compact latent codes $\mathbf{z} \in \mathbb{R}^d$. The CAE is trained on snapshot data and then *frozen*.

2. **Stage 2 — Latent MLP (this network):** A small Multi-Layer Perceptron (MLP) learns to predict the latent codes $\mathbf{z}$ directly from physical parameters (μ, t), bypassing the expensive encoder at inference time.

At **inference** the pipeline is:

$$(\mu,\, t) \xrightarrow{\text{MLP}} \hat{\mathbf{z}} \xrightarrow{\text{Decoder}} \hat{\mathbf{u}}(\mathbf{x})$$

where μ is the physical parameter (e.g. wave speed) and t is the time instant.

2 Input: Physical Parameters (μ, t)

Each snapshot in the database is uniquely identified by a simulation case (parametrized by μ) and a time step t. The MLP receives these as a single input vector:

$$\mathbf{x} = \begin{pmatrix} \mu \\ t \end{pmatrix} \in \mathbb{R}^2$$

More generally, if the PDE has p parameters $(\mu_1, \ldots, \mu_p)$, the input dimension is $p{+}1$ (parameters plus time):

$$\mathbf{x} = (\mu_1,\ \mu_2,\ \ldots,\ \mu_p,\ t)^\top \in \mathbb{R}^{p+1}.$$

Input normalization (z-score). Before entering the network, each input column is standardized using training-set statistics:

$$\hat{x}_j = \frac{x_j - \bar{x}_j}{\sigma_{x_j}}, \qquad j = 1, \ldots, p+1$$

where $\bar{x}_j$ and σ_{x_j} are the mean and standard deviation of column j computed over the training set only. This ensures that every input feature has approximately zero mean and unit variance, which helps gradient-based optimization converge faster and more stably.

3 Network Architecture

The MLP has a simple feedforward architecture with two hidden layers:

$$\mathbf{x} \in \mathbb{R}^{n_{\text{in}}} \xrightarrow{\mathbf{W}^{(1)},\, \mathbf{b}^{(1)}} \mathbf{h}^{(1)} \in \mathbb{R}^{H} \xrightarrow{\sigma} \mathbf{h}^{(1)}_{\text{act}} \xrightarrow{\mathbf{W}^{(2)},\, \mathbf{b}^{(2)}} \mathbf{h}^{(2)} \in \mathbb{R}^{H} \xrightarrow{\sigma} \mathbf{h}^{(2)}_{\text{act}} \xrightarrow{\mathbf{W}^{(3)},\, \mathbf{b}^{(3)}} \hat{\mathbf{z}} \in \mathbb{R}^{d}$$

where $n_{\text{in}} = p{+}1$ is the input dimension, H is the hidden dimension, d is the latent dimension, and σ denotes the ReLU activation function.

3.1 Layer-by-Layer Breakdown

Layer 1 — Linear transformation. The normalized input $\hat{\mathbf{x}} \in \mathbb{R}^{n_{\text{in}}}$ is multiplied by a weight matrix and shifted by a bias vector:

$$\mathbf{h}^{(1)} = \mathbf{W}^{(1)}\hat{\mathbf{x}} + \mathbf{b}^{(1)}, \qquad \mathbf{W}^{(1)} \in \mathbb{R}^{H \times n_{\text{in}}},\ \mathbf{b}^{(1)} \in \mathbb{R}^{H}.$$

This is a standard *affine transformation*: every output neuron computes a weighted sum of all inputs plus a bias. With $n_{\text{in}} = 2$ and $H = 64$ this layer has $2 \times 64 + 64 = 192$ learnable parameters.

Activation 1 — ReLU. The Rectified Linear Unit applies an element-wise nonlinearity:

$$\mathbf{h}^{(1)}_{\text{act}} = \text{ReLU}\big(\mathbf{h}^{(1)}\big) = \max\big(0,\ \mathbf{h}^{(1)}\big).$$

For each component h_i: if $h_i > 0$ it passes through unchanged; if $h_i \leq 0$ it is set to zero. Key properties:

- Introduces **nonlinearity** — without it, stacking linear layers would collapse to a single linear map, and the network could not approximate nonlinear functions.
- Computationally cheap (just a threshold).
- Gradients are either 0 or 1, avoiding the vanishing gradient problem of sigmoid/tanh for deep networks.
- **No learnable parameters** — it is a fixed pointwise operation.

Layer 2 — Linear transformation.

$$\mathbf{h}^{(2)} = \mathbf{W}^{(2)}\mathbf{h}^{(1)}_{\text{act}} + \mathbf{b}^{(2)}, \qquad \mathbf{W}^{(2)} \in \mathbb{R}^{H \times H},\ \mathbf{b}^{(2)} \in \mathbb{R}^{H}.$$

With $H = 64$: $64 \times 64 + 64 = 4{,}160$ parameters.

Activation 2 — ReLU.

$$\mathbf{h}^{(2)}_{\text{act}} = \text{ReLU}\big(\mathbf{h}^{(2)}\big).$$

Same pointwise operation as before, applied to the second hidden layer.

Layer 3 — Output linear transformation.

$$\hat{\mathbf{z}} = \mathbf{W}^{(3)}\mathbf{h}^{(2)}_{\text{act}} + \mathbf{b}^{(3)}, \qquad \mathbf{W}^{(3)} \in \mathbb{R}^{d \times H},\ \mathbf{b}^{(3)} \in \mathbb{R}^{d}.$$

With $H = 64$ and $d = 4$: $64 \times 4 + 4 = 260$ parameters.

No output activation is applied. The latent codes $\mathbf{z}$ produced by the encoder can take any real value, so the MLP output must be unconstrained.

Table 1: MLP architecture summary (default: $n_{\text{in}} = 2$, $H = 64$, $d = 4$).

Layer	Operation	Input dim	Output dim	Parameters
1	$\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$	n_{in}	H	$n_{\text{in}} \cdot H + H$
	ReLU	H	H	0
2	$\mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$	H	H	$H^2 + H$
	ReLU	H	H	0
3	$\mathbf{W}^{(3)}\mathbf{h} + \mathbf{b}^{(3)}$	H	d	$H \cdot d + d$
Total (CNN encoder, $d = 4$**)**				4,612
Total (KTH encoder, $d = 1024$**)**				70,848

3.2 Summary Table

The total parameter count is:

$$N_{\text{params}} = n_{\text{in}} \cdot H + H + H^2 + H + H \cdot d + d = (n_{\text{in}} + H + d)\, H + (H + d).$$

3.3 Who Decides These Numbers? Hyperparameters vs. Architecture

A common source of confusion: *"Where does 64 come from? And is the network 3 layers or 2?"*

Hidden dimension $H = 64$. This is a **hyperparameter** — a design choice made by the user *before* training, not something the network learns. It is set in `train_latent_mlp.py`:

```
HIDDEN_DIM = 64
```

There is no formula that dictates $H = 64$. The value is chosen by the practitioner based on experience, problem complexity, and experimentation. Common considerations:

- **Too small** ($H = 4$): The network cannot capture the nonlinear mapping and underfits.
- **Too large** ($H = 1024$): The network has far more capacity than needed for a $\mathbb{R}^2 \to \mathbb{R}^d$ mapping, wastes memory, and may overfit.
- $H = 64$ is a reasonable middle ground for a smooth, low-dimensional regression problem. It could equally have been 32 or 128 — the choice is empirical.

How many layers? The "3 layers" question. The terminology can be confusing because there are two common counting conventions:

Counting method	Count	What is counted
Linear layers (weight matrices)	**3**	$\mathbf{W}^{(1)}$, $\mathbf{W}^{(2)}$, $\mathbf{W}^{(3)}$
Hidden layers (internal representations)	**2**	$\mathbf{h}^{(1)}_{\text{act}}$, $\mathbf{h}^{(2)}_{\text{act}}$

Both are correct — they count different things:

- When people say "a **3-layer MLP**," they usually mean 3 linear (weight) layers: input→hidden1, hidden1→hidden2, hidden2→output.
- When people say "**2 hidden layers**," they mean 2 intermediate representations that are neither the input nor the output.
- The input $\hat{\mathbf{x}}$ and the output $\hat{\mathbf{z}}$ are *not* counted as hidden layers.

Visually:

$$\underbrace{\hat{\mathbf{x}}}_{\text{input}} \xrightarrow{\mathbf{W}^{(1)}} \underbrace{\mathbf{h}_{\text{act}}^{(1)}}_{\text{hidden 1}} \xrightarrow{\mathbf{W}^{(2)}} \underbrace{\mathbf{h}_{\text{act}}^{(2)}}_{\text{hidden 2}} \xrightarrow{\mathbf{W}^{(3)}} \underbrace{\hat{\mathbf{z}}}_{\text{output}}$$

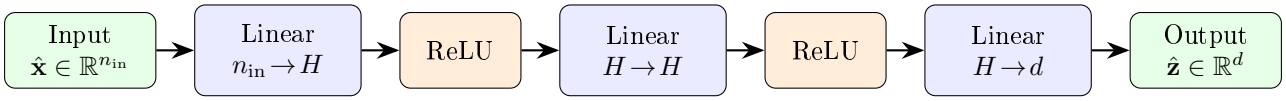
$$\text{3 weight matrices} \quad = \quad \text{3 linear layers} \quad = \quad \text{2 hidden layers}$$

So this MLP has **3 linear layers** (equivalently: **2 hidden layers**). The number 3 refers to weight matrices; the number 2 refers to hidden representations.

Hyperparameter	**Symbol**	**Value**	**Set by**
Hidden dimension	H	64	User (`HIDDEN_DIM`)
Number of hidden layers	—	2	Hardcoded in model
Latent dimension	d	4 or 1024	Determined by Stage-1 encoder
Input dimension	n_{in}	$p+1$	From dataset (p params + time)
Learning rate	η	10^{-3}	User (`LEARNING_RATE`)
Batch size	—	32	User (`BATCH_SIZE`)

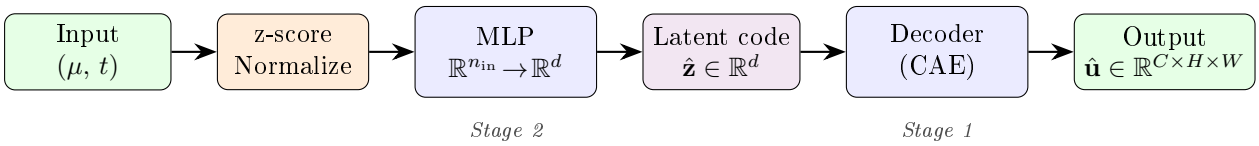
Summary of all user-chosen hyperparameters. The number of hidden layers (2) and the activation function (ReLU) are hardcoded in the `LatentMLP` class. To change them, one would modify the model code in `latent_mlp/models/latent_mlp.py`.

4 Network Diagram



4.1 Full Prediction Pipeline

At inference, the entire CAE + MLP pipeline maps physical parameters (μ, t) to a reconstructed spatial field $\hat{\mathbf{u}} \in \mathbb{R}^{C \times H \times W}$:



The encoder is *not* used at inference — the MLP replaces it entirely, producing the latent code $\hat{\mathbf{z}}$ directly from the physical parameters. Only the frozen decoder is needed to reconstruct the full spatial field.

5 Training Procedure

5.1 Target Generation

The MLP is *not* trained on raw field data. Instead, the frozen Stage-1 encoder produces the training targets:

1. Load all training snapshots $\mathbf{u}_i$ from HDF5.

2. (Optionally) apply per-snapshot min–max normalization to $[0, 1]$ if the encoder was trained with normalization.

3. Pass through the frozen encoder: $\mathbf{z}_i = \text{Encoder}(\mathbf{u}_i) \in \mathbb{R}^d$.

4. For the KTH encoder, the latent is a spatial map $\mathbf{z} \in \mathbb{R}^{z_c \times H' \times W'}$ which is flattened to a vector $\mathbf{z} \in \mathbb{R}^{z_c \cdot H' \cdot W'}$.

The MLP training data is then $\{(\hat{\mathbf{x}}_i,\ \mathbf{z}_i)\}_{i=1}^{N_{\text{train}}}$.

5.2 Loss Function

The MLP is trained to minimize the Mean Squared Error (MSE) between predicted and true latent codes:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \|\hat{\mathbf{z}}_i - \mathbf{z}_i\|_2^2 = \frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{d} \big(\hat{z}_{ij} - z_{ij}\big)^2.$$

5.3 Optimizer and Learning Rate Schedule

- **Optimizer:** Adam with default momentum parameters ($\beta_1 = 0.9,\ \beta_2 = 0.999$) and initial learning rate $\eta = 10^{-3}$.
- **Scheduler:** `ReduceLROnPlateau` — if the validation loss does not decrease for 100 consecutive epochs, the learning rate is halved ($\eta \leftarrow 0.5\,\eta$).
- **Batch size:** 32.

5.4 Early Stopping

Training runs for up to `MAX_EPOCHS` (5,000) epochs. If the validation loss does not improve for `PATIENCE` (500) consecutive epochs, training stops early. The model weights from the best validation epoch are saved as the final model.

6 Configurations Used in This Project

Table 2: MLP configurations for the two encoder types.

Setting	**CNN (simple_cae)**	**KTH (resnet_cae)**
Input dim (n_{in})	2 (μ, t)	2 (μ, t)
Hidden dim (H)	64	64
Latent dim (d)	4	1,024
Total parameters	4,612	70,848
Learning rate	10^{-3}	10^{-3}
Batch size	32	32
Max epochs	5,000	5,000
Patience	500	500

The large difference in latent dimension (4 vs. 1,024) is because the CNN encoder compresses to a flat 4-element vector, while the KTH encoder uses a spatial latent map that is flattened for the MLP.

7 What Each Component Does — Intuition

The linear layers are learnable affine transformations. Each neuron in a linear layer computes:

$$h_j = \sum_{i=1}^{n} w_{ji}\, x_i + b_j$$

This is a weighted sum (like fitting a hyperplane). A single linear layer can only represent linear relationships between inputs and outputs.

The ReLU activations introduce nonlinearity by zeroing out negative values. This is critical: without nonlinear activations, stacking any number of linear layers would still be equivalent to a single linear transformation ($\mathbf{W}^{(3)}\mathbf{W}^{(2)}\mathbf{W}^{(1)}\mathbf{x}+\ldots$). The ReLU creates piecewise-linear decision regions, enabling the network to approximate complex nonlinear mappings.

Together, the network partitions the input space into regions (defined by which ReLU neurons are active or inactive) and fits a different linear function in each region. With 64 hidden neurons per layer, the network can create up to 2^{64} distinct linear regions in theory — far more than enough to approximate the smooth mapping $(\mu, t) \to \mathbf{z}$.

Why no output activation? The target latent codes $\mathbf{z}$ from the encoder are unbounded real numbers. Applying an activation like sigmoid or tanh at the output would clamp predictions to a fixed range, preventing the MLP from matching the true latent values.

Why is the MLP so small? The mapping $(\mu, t) \to \mathbf{z}$ is typically smooth and low-dimensional ($\mathbb{R}^2 \to \mathbb{R}^d$). A small MLP with ~5,000 parameters is sufficient. The heavy lifting (high-dimensional spatial reconstruction) is done by the decoder, which has millions of parameters.

8 Numerical Worked Example

We trace a *single* input through the entire network with concrete numbers. To keep the arithmetic readable we use a **toy size**: $n_{\text{in}} = 2$, $H = 3$ (hidden), $d = 2$ (latent). The real network uses $H = 64$ — the operations are identical, just larger matrices.

8.1 Step 0 — Raw physical parameters

Suppose we want to predict the latent code for wave speed $\mu = 1.8$ at time $t = 0.6$. The raw input is:

$$\mathbf{x}_{\text{raw}} = \begin{pmatrix} 1.8 \\ 0.6 \end{pmatrix}.$$

8.2 Step 1 — Input normalization (z-score)

During training, we computed the mean and standard deviation of each input column. Suppose:

$$\bar{\mathbf{x}} = \begin{pmatrix} 1.5 \\ 0.5 \end{pmatrix}, \qquad \boldsymbol{\sigma} = \begin{pmatrix} 0.3 \\ 0.2 \end{pmatrix}.$$

The normalized input is computed element-wise:

$$\hat{x}_1 = \frac{1.8 - 1.5}{0.3} = 1.0, \qquad \hat{x}_2 = \frac{0.6 - 0.5}{0.2} = 0.5.$$

$$\boxed{\hat{\mathbf{x}} = \begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}}$$

This is what actually enters the network. Every input feature now has a similar scale, regardless of its physical units.

8.3 Step 2 — Layer 1: Linear $(2 \to 3)$

The first linear layer has a weight matrix $\mathbf{W}^{(1)} \in \mathbb{R}^{3\times 2}$ and bias $\mathbf{b}^{(1)} \in \mathbb{R}^3$. Suppose the trained weights are:

$$\mathbf{W}^{(1)} = \begin{pmatrix} 0.6 & -0.4 \\ -0.3 & 0.8 \\ 0.5 & 0.5 \end{pmatrix}, \qquad \mathbf{b}^{(1)} = \begin{pmatrix} 0.1 \\ -0.2 \\ 0.0 \end{pmatrix}.$$

Each row of $\mathbf{W}^{(1)}$ defines one hidden neuron. The computation is:

$$h_1^{(1)} = w_{11}\,\hat{x}_1 + w_{12}\,\hat{x}_2 + b_1 = (0.6)(1.0) + (-0.4)(0.5) + 0.1 = 0.6 - 0.2 + 0.1 = \mathbf{0.5},$$

$$h_2^{(1)} = w_{21}\,\hat{x}_1 + w_{22}\,\hat{x}_2 + b_2 = (-0.3)(1.0) + (0.8)(0.5) + (-0.2) = -0.3 + 0.4 - 0.2 = \mathbf{-0.1},$$

$$h_3^{(1)} = w_{31}\,\hat{x}_1 + w_{32}\,\hat{x}_2 + b_3 = (0.5)(1.0) + (0.5)(0.5) + 0.0 = 0.5 + 0.25 = \mathbf{0.75}.$$

In matrix form:

$$\mathbf{h}^{(1)} = \mathbf{W}^{(1)}\hat{\mathbf{x}} + \mathbf{b}^{(1)} = \begin{pmatrix} 0.5 \\ -0.1 \\ 0.75 \end{pmatrix}.$$

8.4 Step 3 — Activation 1: ReLU

ReLU is applied element-wise: $\max(0,\ h_j)$.

$$\begin{aligned} \mathrm{ReLU}(0.5) &= \max(0,\ 0.5) = \mathbf{0.5} && \text{(positive} \to \text{passes through)}, \\ \mathrm{ReLU}(-0.1) &= \max(0,\ -0.1) = \mathbf{0.0} && \text{(negative} \to \text{killed to zero)}, \\ \mathrm{ReLU}(0.75) &= \max(0,\ 0.75) = \mathbf{0.75} && \text{(positive} \to \text{passes through)}. \end{aligned}$$

$$\boxed{\mathbf{h}_{\text{act}}^{(1)} = \begin{pmatrix} 0.5 \\ 0.0 \\ 0.75 \end{pmatrix}}$$

Key observation: Neuron 2 is "dead" for this input — its output is zero and it contributes nothing to downstream computations. Different inputs will activate different subsets of neurons, creating the piecewise-linear behavior discussed earlier.

8.5 Step 4 — Layer 2: Linear $(3 \to 3)$

The second linear layer has $\mathbf{W}^{(2)} \in \mathbb{R}^{3\times 3}$ and $\mathbf{b}^{(2)} \in \mathbb{R}^3$. Suppose:

$$\mathbf{W}^{(2)} = \begin{pmatrix} 0.4 & 0.2 & -0.6 \\ -0.5 & 0.7 & 0.3 \\ 0.1 & -0.3 & 0.9 \end{pmatrix}, \qquad \mathbf{b}^{(2)} = \begin{pmatrix} 0.05 \\ 0.10 \\ -0.05 \end{pmatrix}.$$

Since $h_{\text{act},2}^{(1)} = 0$, the second column of $\mathbf{W}^{(2)}$ is multiplied by zero and has no effect:

$$h_1^{(2)} = (0.4)(0.5) + (0.2)(0.0) + (-0.6)(0.75) + 0.05 = 0.2 + 0 - 0.45 + 0.05 = \mathbf{-0.2},$$

$$h_2^{(2)} = (-0.5)(0.5) + (0.7)(0.0) + (0.3)(0.75) + 0.10 = -0.25 + 0 + 0.225 + 0.10 = \mathbf{0.075},$$

$$h_3^{(2)} = (0.1)(0.5) + (-0.3)(0.0) + (0.9)(0.75) + (-0.05) = 0.05 + 0 + 0.675 - 0.05 = \mathbf{0.675}.$$

$$\mathbf{h}^{(2)} = \begin{pmatrix} -0.2 \\ 0.075 \\ 0.675 \end{pmatrix}.$$

8.6 Step 5 — Activation 2: ReLU

$$\begin{aligned} \mathrm{ReLU}(-0.2) &= \mathbf{0.0} \qquad \text{(killed)}, \\ \mathrm{ReLU}(0.075) &= \mathbf{0.075} \qquad \text{(passes)}, \\ \mathrm{ReLU}(0.675) &= \mathbf{0.675} \qquad \text{(passes)}. \end{aligned}$$

$$\boxed{\mathbf{h}^{(2)}_{\text{act}} = \begin{pmatrix} 0.0 \\ 0.075 \\ 0.675 \end{pmatrix}}$$

Now neuron 1 is dead (different from Layer 1 where neuron 2 was dead). This shows how the "active region" changes from layer to layer.

8.7 Step 6 — Layer 3: Output Linear ($3 \to 2$)

The final layer maps to the latent dimension $d = 2$. $\mathbf{W}^{(3)} \in \mathbb{R}^{2\times 3}$, $\mathbf{b}^{(3)} \in \mathbb{R}^2$. Suppose:

$$\mathbf{W}^{(3)} = \begin{pmatrix} 0.3 & -0.8 & 1.2 \\ 0.7 & 0.4 & -0.5 \end{pmatrix}, \qquad \mathbf{b}^{(3)} = \begin{pmatrix} 0.02 \\ -0.01 \end{pmatrix}.$$

$$\hat{z}_1 = (0.3)(0.0) + (-0.8)(0.075) + (1.2)(0.675) + 0.02 = 0 - 0.06 + 0.81 + 0.02 = \mathbf{0.77},$$

$$\hat{z}_2 = (0.7)(0.0) + (0.4)(0.075) + (-0.5)(0.675) + (-0.01) = 0 + 0.03 - 0.3375 - 0.01 = \mathbf{-0.3175}.$$

$$\boxed{\hat{\mathbf{z}} = \begin{pmatrix} 0.77 \\ -0.3175 \end{pmatrix}} \quad \longleftarrow \text{ predicted latent code (no activation!)}$$

Note that $\hat{z}_2$ is *negative* — this is perfectly fine because there is no output activation to clamp the values.

8.8 Complete Forward Pass Summary

$$\underbrace{\begin{pmatrix} 1.8 \\ 0.6 \end{pmatrix}}_{\text{raw } (\mu,t)} \xrightarrow{\text{z-score}} \underbrace{\begin{pmatrix} 1.0 \\ 0.5 \end{pmatrix}}_{\hat{\mathbf{x}}} \xrightarrow{\mathbf{W}^{(1)}\hat{\mathbf{x}}+\mathbf{b}^{(1)}} \underbrace{\begin{pmatrix} 0.5 \\ -0.1 \\ 0.75 \end{pmatrix}}_{\mathbf{h}^{(1)}} \xrightarrow{\text{ReLU}} \underbrace{\begin{pmatrix} 0.5 \\ 0.0 \\ 0.75 \end{pmatrix}}_{\mathbf{h}^{(1)}_{\text{act}}}$$

$$\xrightarrow{\mathbf{W}^{(2)}\mathbf{h}+\mathbf{b}^{(2)}} \underbrace{\begin{pmatrix} -0.2 \\ 0.075 \\ 0.675 \end{pmatrix}}_{\mathbf{h}^{(2)}} \xrightarrow{\text{ReLU}} \underbrace{\begin{pmatrix} 0.0 \\ 0.075 \\ 0.675 \end{pmatrix}}_{\mathbf{h}^{(2)}_{\text{act}}} \xrightarrow{\mathbf{W}^{(3)}\mathbf{h}+\mathbf{b}^{(3)}} \boxed{\hat{\mathbf{z}} = \begin{pmatrix} 0.77 \\ -0.3175 \end{pmatrix}}$$

8.9 What Happens Next

This predicted latent code $\hat{\mathbf{z}}$ is passed to the Stage-1 **Decoder** to reconstruct the full spatial field:

$$\hat{\mathbf{z}} = \begin{pmatrix} 0.77 \\ -0.3175 \end{pmatrix} \xrightarrow{\text{Decoder}} \hat{\mathbf{u}} \in \mathbb{R}^{1\times 256\times 256}$$

The decoder (a large convolutional network with millions of parameters) maps this tiny 2-element vector back to the 256×256 field — that is the power of the learned latent representation.

8.10 Scaling to the Real Network

In the actual network ($H = 64$, $d = 4$), the operations are identical:

- Layer 1 multiplies a (64×2) matrix by the 2-element input $\rightarrow$ 64 pre-activations, then ReLU zeros out some of the 64.
- Layer 2 multiplies a (64×64) matrix by the 64-element vector $\rightarrow$ 64 pre-activations, then ReLU zeros out some.
- Layer 3 multiplies a (4×64) matrix by the 64-element vector $\rightarrow$ the final 4 latent code values.

The only difference is the size of the matrices — the algebra is the same as shown above, just with larger vectors.